

Python For Data Science Cheat Sheet

Pandas Basics

Learn Python for Data Science Interactively at [www.DataCamp.com](https://www.datacamp.com)



Pandas

The **Pandas** library is built on NumPy and provides easy-to-use **data structures** and **data analysis tools** for the Python programming language.



Use the following import convention:

```
>>> import pandas as pd
```

Pandas Data Structures

Series

A **one-dimensional** labeled array capable of holding any data type

a	3
b	-5
c	7
d	4

Index

```
>>> s = pd.Series([3, -5, 7, 4], index=['a', 'b', 'c', 'd'])
```

DataFrame

	Country	Capital	Population
0	Belgium	Brussels	11190846
1	India	New Delhi	1303171035
2	Brazil	Brasília	207847528

A **two-dimensional** labeled data structure with columns of potentially different types

```
>>> data = {'Country': ['Belgium', 'India', 'Brazil'],
           'Capital': ['Brussels', 'New Delhi', 'Brasília'],
           'Population': [11190846, 1303171035, 207847528]}
```

```
>>> df = pd.DataFrame(data,
                      columns=['Country', 'Capital', 'Population'])
```

I/O

Read and Write to CSV

```
>>> pd.read_csv('file.csv', header=None, nrows=5)
>>> df.to_csv('myDataFrame.csv')
```

Read and Write to Excel

```
>>> pd.read_excel('file.xlsx')
>>> pd.to_excel('dir/myDataFrame.xlsx', sheet_name='Sheet1')

Read multiple sheets from the same file
>>> xlsx = pd.ExcelFile('file.xls')
>>> df = pd.read_excel(xlsx, 'Sheet1')
```

Asking For Help

```
>>> help(pd.Series.loc)
```

Selection

Also see NumPy Arrays

Getting

```
>>> s['b']
-5
```

Get one element

```
>>> df[1:]
   Country  Capital  Population
1   India  New Delhi  1303171035
2  Brazil  Brasília  207847528
```

Get subset of a DataFrame

Selecting, Boolean Indexing & Setting

By Position

```
>>> df.iloc([0], [0])
```

Select single value by row & column

```
'Belgium'
```

```
>>> df.iat([0], [0])
```

```
'Belgium'
```

By Label

```
>>> df.loc([0], ['Country'])
```

Select single value by row & column labels

```
'Belgium'
```

```
>>> df.at([0], ['Country'])
```

```
'Belgium'
```

By Label/Position

```
>>> df.ix[2]
```

Select single row of subset of rows

```
Country      Brazil
Capital      Brasília
Population    207847528
```

```
>>> df.ix[:, 'Capital']
```

Select a single column of subset of columns

```
0    Brussels
1    New Delhi
2    Brasilia
```

```
>>> df.ix[1, 'Capital']
```

Select rows and columns

```
'New Delhi'
```

Boolean Indexing

```
>>> s[~(s > 1)]
```

Series *s* where value is not >1

```
>>> s[(s < -1) | (s > 2)]
```

s where value is <-1 or >2

```
>>> df[df['Population'] > 1200000000]
```

Use filter to adjust DataFrame

Setting

```
>>> s['a'] = 6
```

Set index *a* of Series *s* to 6

Dropping

```
>>> s.drop(['a', 'c'])
```

Drop values from rows (axis=0)

```
>>> df.drop('Country', axis=1)
```

Drop values from columns(axis=1)

Sort & Rank

```
>>> df.sort_index()
>>> df.sort_values(by='Country')
>>> df.rank()
```

Sort by labels along an axis
Sort by the values along an axis
Assign ranks to entries

Retrieving Series/DataFrame Information

Basic Information

```
>>> df.shape
>>> df.index
>>> df.columns
>>> df.info()
>>> df.count()
```

(rows,columns)
Describe index
Describe DataFrame columns
Info on DataFrame
Number of non-NA values

Summary

```
>>> df.sum()
>>> df.cumsum()
>>> df.min()/df.max()
>>> df.idxmin()/df.idxmax()
>>> df.describe()
>>> df.mean()
>>> df.median()
```

Sum of values
Cumulative sum of values
Minimum/maximum values
Minimum/Maximum index value
Summary statistics
Mean of values
Median of values

Applying Functions

```
>>> f = lambda x: x*2
>>> df.apply(f)
>>> df.applymap(f)
```

Apply function
Apply function element-wise

Data Alignment

Internal Data Alignment

NA values are introduced in the indices that don't overlap:

```
>>> s3 = pd.Series([7, -2, 3], index=['a', 'c', 'd'])
>>> s + s3
a      10.0
b      NaN
c       5.0
d       7.0
```

Arithmetic Operations with Fill Methods

You can also do the internal data alignment yourself with the help of the fill methods:

```
>>> s.add(s3, fill_value=0)
a      10.0
b     -5.0
c       5.0
d       7.0
>>> s.sub(s3, fill_value=2)
>>> s.div(s3, fill_value=4)
>>> s.mul(s3, fill_value=3)
```

Read and Write to SQL Query or Database Table

```
>>> from sqlalchemy import create_engine
>>> engine = create_engine('sqlite:///memory:')
>>> pd.read_sql("SELECT * FROM my_table;", engine)
>>> pd.read_sql_table('my_table', engine)
>>> pd.read_sql_query("SELECT * FROM my_table;", engine)
```

`read_sql()` is a convenience wrapper around `read_sql_table()` and `read_sql_query()`

```
>>> pd.to_sql('myDf', engine)
```

DataCamp

Learn Python for Data Science Interactively



Python For Data Science Cheat Sheet

Pandas

Learn Python for Data Science Interactively at [www.DataCamp.com](https://www.datacamp.com)



Reshaping Data

Pivot

```
>>> df3= df2.pivot(index='Date',
                    columns='Type',
                    values='Value')
```

Spread rows into columns

	Date	Type	Value
0	2016-03-01	a	11.432
1	2016-03-02	b	13.031
2	2016-03-01	c	20.784
3	2016-03-03	a	99.906
4	2016-03-02	a	1.303
5	2016-03-03	c	20.784

Type	a	b	c
Date			
2016-03-01	11.432	NaN	20.784
2016-03-02	1.303	13.031	NaN
2016-03-03	99.906	NaN	20.784

Pivot Table

```
>>> df4 = pd.pivot_table(df2,
                        values='Value',
                        index='Date',
                        columns='Type')
```

Spread rows into columns

Stack / Unstack

```
>>> stacked = df5.stack()
>>> stacked.unstack()
```

Pivot a level of column labels
Pivot a level of index labels

	0	1
1	5	0.233482
2	4	0.184713
3	3	0.433522

Unstacked

	0	1	2
1	5	0	0.233482
2	4	0	0.184713
3	3	0	0.433522
4	1	1	0.429401

Stacked

Melt

```
>>> pd.melt(df2,
            id_vars=["Date"],
            value_vars=["Type", "Value"],
            value_name="Observations")
```

Gather columns into rows

	Date	Type	Value
0	2016-03-01	a	11.432
1	2016-03-02	b	13.031
2	2016-03-01	c	20.784
3	2016-03-03	a	99.906
4	2016-03-02	a	1.303
5	2016-03-03	c	20.784

	Date	Variable	Observations
0	2016-03-01	Type	a
1	2016-03-02	Type	b
2	2016-03-01	Type	c
3	2016-03-03	Type	a
4	2016-03-02	Type	a
5	2016-03-03	Type	c
6	2016-03-01	Value	11.432
7	2016-03-02	Value	13.031
8	2016-03-01	Value	20.784
9	2016-03-03	Value	99.906
10	2016-03-02	Value	1.303
11	2016-03-03	Value	20.784

Iteration

```
>>> df.iteritems()
>>> df.iterrows()
```

(Column-index, Series) pairs
(Row-index, Series) pairs

Advanced Indexing

Also see NumPy Arrays

Selecting

```
>>> df3.loc[:, (df3>1).any()]
>>> df3.loc[:, (df3>1).all()]
>>> df3.loc[:, df3.isnull().any()]
>>> df3.loc[:, df3.notnull().all()]
```

Select cols with any vals >1
Select cols with vals > 1
Select cols with NaN
Select cols without NaN

Indexing With isin

```
>>> df[(df.Country.isin(df2.Type))]
>>> df3.filter(items=["a", "b"])
>>> df.select(lambda x: not x%5)
```

Find same elements
Filter on values
Select specific elements

Where

```
>>> s.where(s > 0)
```

Subset the data

Query

```
>>> df6.query('second > first')
```

Query DataFrame

Setting/Resetting Index

```
>>> df.set_index('Country')
>>> df4 = df.reset_index()
>>> df = df.rename(index=str,
                  columns={"Country": "centry",
                           "Capital": "cptl",
                           "Population": "ppltn"})
```

Set the index
Reset the index
Rename DataFrame

Reindexing

```
>>> s2 = s.reindex(['a', 'c', 'd', 'e', 'b'])
```

Forward Filling

```
>>> df.reindex(range(4),
               method='ffill')
Country Capital Population
0 Belgium Brussels 11190846
1 India New Delhi 1303171035
2 Brazil Brasilia 207847528
3 Brazil Brasilia 207847528
```

Backward Filling

```
>>> s3 = s.reindex(range(5),
                   method='bfill')
0 3
1 3
2 3
3 3
4 3
```

MultiIndexing

```
>>> arrays = [np.array([1,2,3]),
              np.array([5,4,3])]
>>> df5 = pd.DataFrame(np.random.rand(3, 2), index=arrays)
>>> tuples = list(zip(*arrays))
>>> index = pd.MultiIndex.from_tuples(tuples,
                                     names=['first', 'second'])
>>> df6 = pd.DataFrame(np.random.rand(3, 2), index=index)
>>> df2.set_index(["Date", "Type"])
```

Duplicate Data

```
>>> s3.unique()
>>> df2.duplicated('Type')
>>> df2.drop_duplicates('Type', keep='last')
>>> df.index.duplicated()
```

Return unique values
Check duplicates
Drop duplicates
Check index duplicates

Grouping Data

Aggregation

```
>>> df2.groupby(by=['Date', 'Type']).mean()
>>> df4.groupby(level=0).sum()
>>> df4.groupby(level=0).agg({'a': lambda x: sum(x)/len(x),
                           'b': np.sum})
```

Transformation

```
>>> customSum = lambda x: (x+x%2)
>>> df4.groupby(level=0).transform(customSum)
```

Missing Data

```
>>> df.dropna()
>>> df3.fillna(df3.mean())
>>> df2.replace("a", "f")
```

Drop NaN values
Fill NaN values with a predetermined value
Replace values with others

Combining Data

data1		data2	
X1	X2	X1	X3
a	11.432	a	20.784
b	1.303	b	NaN
c	99.906	d	20.784

Merge

```
>>> pd.merge(data1,
             data2,
             how='left',
             on='X1')
```

X1	X2	X3
a	11.432	20.784
b	1.303	NaN
c	99.906	NaN

```
>>> pd.merge(data1,
             data2,
             how='right',
             on='X1')
```

X1	X2	X3
a	11.432	20.784
b	1.303	NaN
d	NaN	20.784

```
>>> pd.merge(data1,
             data2,
             how='inner',
             on='X1')
```

X1	X2	X3
a	11.432	20.784
b	1.303	NaN

```
>>> pd.merge(data1,
             data2,
             how='outer',
             on='X1')
```

X1	X2	X3
a	11.432	20.784
b	1.303	NaN
c	99.906	NaN
d	NaN	20.784

Join

```
>>> data1.join(data2, how='right')
```

Concatenate

Vertical

```
>>> s.append(s2)
```

Horizontal/Vertical

```
>>> pd.concat([s,s2],axis=1, keys=['One', 'Two'])
>>> pd.concat([data1, data2], axis=1, join='inner')
```

Dates

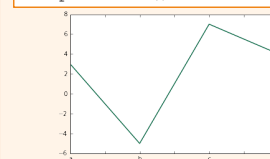
```
>>> df2['Date']= pd.to_datetime(df2['Date'])
>>> df2['Date']= pd.date_range('2000-1-1',
                             periods=6,
                             freq='M')
>>> dates = [datetime(2012,5,1), datetime(2012,5,2)]
>>> index = pd.DatetimeIndex(dates)
>>> index = pd.date_range(datetime(2012,2,1), end, freq='BM')
```

Visualization

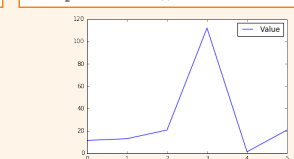
Also see Matplotlib

```
>>> import matplotlib.pyplot as plt
```

```
>>> s.plot()
>>> plt.show()
```



```
>>> df2.plot()
>>> plt.show()
```



DataCamp

Learn Python for Data Science Interactively



Data Analysis with PANDAS

CHEAT SHEET

CREATED BY: ARIANNE COLTON AND SEAN CHEN

DATA STRUCTURES

SERIES (1D)

One-dimensional array-like object containing an array of data (of any **Numpy** data type) and an associated array of data labels, called its “**index**”. If index of data is not specified, then a default one consisting of the integers 0 through N-1 is created.

Create Series	<pre>series1 = pd.Series ([1, 2], index = ['a', 'b']) series1 = pd.Series(dict1) *</pre>
Get Series Values	<pre>series1.values</pre>
Get Values by Index	<pre>series1['a'] series1[['b', 'a']]</pre>
Get Series Index	<pre>series1.index</pre>
Get Name Attribute (None is default)	<pre>series1.name series1.index.name</pre>
** Common Index Values are Added	<pre>series1 + series2</pre>
Unique But Unsorted	<pre>series2 = series1.unique ()</pre>

* Can think of Series as a fixed-length, ordered dict. Series can be substituted into many functions that expect a dict.

** Auto-align differently-indexed data in arithmetic operations

DATAFRAME (2D)

Tabular data structure with ordered collections of columns, each of which can be different value type. Data Frame (DF) can be thought of as a dict of Series.

Create DF (from a dict of equal-length lists or Numpy arrays)	<pre>dict1 = {'state': ['Ohio', 'CA'], 'year': [2000, 2010]} df1 = pd.DataFrame(dict1) # columns are placed in sorted order df1 = pd.DataFrame(dict1, index = ['row1', 'row2']) # specifying index df1 = pd.DataFrame(dict1, columns = ['year', 'state']) # columns are placed in your given order</pre>
* Create DF (from nested dict of dicts)	<pre>dict1 = {'coll': {'row1': 1, 'row2': 2}, 'col2': {'row1': 3, 'row2': 4}}</pre>
The inner keys as row indices	<pre>df1 = pd.DataFrame(dict1)</pre>

Get Columns and Row Names	<pre>df1.columns df1.index</pre>
Get Name Attribute (None is default)	<pre>df1.columns.name df1.index.name</pre>
Get Values	<pre># returns the data as a 2D ndarray, the dtype will be chosen to accomodate all of the columns</pre>
** Get Column as Series	<pre>df1['state'] or df1.state</pre>
** Get Row as Series	<pre>df1.ix['row2'] or df1.ix[1]</pre>
Assign a column that doesn't exist will create a new column	<pre>df1['eastern'] = df1.state == 'Ohio'</pre>
Delete a column	<pre>del df1['eastern']</pre>
Switch Columns and Rows	<pre>df1.T</pre>

* Dicts of Series are treated the same as Nested dict of dicts.

** Data returned is a 'view' on the underlying data, NOT a copy. Thus, any in-place modifications to the data will be reflected in df1.

PANEL DATA (3D)

Create Panel Data : (Each item in the Panel is a DF)

```
import pandas_datareader.data as web
panel1 = pd.Panel({stk : web.get_data_yahoo(stk, '1/1/2000', '1/1/2010')
for stk in ['AAPL', 'IBM']})
# panel1 Dimensions : 2 (item) * 861 (major) * 6 (minor)
```

"Stacked" DF form : (Useful way to represent panel data)

```
panel1 = panel1.swapaxes('item', 'minor')
panel1.ix[:, '6/1/2003', :].to_frame() *
=> Stacked DF (with hierarchical indexing *) :
```

#		Open	High	Low	Close	Volume	Adj-Close
# major	minor						
# 2003-06-01	AAPL						
#	IBM						
# 2003-06-02	AAPL						
#	IBM						

DATA STRUCTURES CONTINUED

- * DF has a “to_panel()” method which is the inverse of “to_frame()”.
- ** Hierarchical indexing makes N-dimensional arrays unnecessary in a lot of cases. Aka prefer to use Stacked DF, not Panel data.

INDEX OBJECTS

Immutable objects that hold the axis labels and other metadata (i.e. axis name)

- i.e. Index, MultiIndex, DatetimeIndex, PeriodIndex
- Any sequence of labels used when constructing Series or DF internally converted to an Index.
- Can functions as fixed-size set in addition to being array-like.

HIERARCHICAL INDEXING

Multiple index levels on an axis : A way to work with higher dimensional data in a lower dimensional form.

MultiIndex :	<pre>series1 = Series(np.random.randn(6), index = [['a', 'a', 'a', 'b', 'b', 'b'], [1, 2, 3, 1, 2, 3]]) series1.index.names = ['key1', 'key2']</pre>
Series Partial Indexing	<pre>series1['b'] # Outer Level series1[:, 2] # Inner Level</pre>
DF Partial Indexing	<pre>df1['outerCol3', 'InnerCol2'] Or df1['outerCol3']['InnerCol2']</pre>

Swapping and Sorting Levels

Swap Level (level interchanged) *	<pre>swapSeries1 = series1.swaplevel('key1', 'key2')</pre>
Sort Level	<pre>series1.sortlevel(1) # sorts according to first inner level</pre>

Common Ops : Swap and Sort **	<pre>series1.swaplevel(0, 1).sortlevel(0) # the order of rows also change</pre>
-------------------------------	---

- * The order of the rows do not change. Only the two levels got swapped.
- ** Data selection performance is much better if the index is sorted starting with the outermost level, as a result of calling `sortlevel(0)` or `sort_index()`.

Summary Statistics by Level

Most stats functions in DF or Series have a “level” option that you can specify the level you want on an axis.

Sum rows (that have same 'key2' value)	<pre>df1.sum(level = 'key2')</pre>
Sum columns ..	<pre>df1.sum(level = 'col3', axis = 1)</pre>

- Under the hood, the functionality provided here utilizes panda's “**groupby**”.

DataFrame's Columns as Indexes

DF's “set_index” will create a new DF using one or more of its columns as the index.

New DF using columns as index	<pre>df2 = df1.set_index(['col3', 'col4']) * ‡ # col3 becomes the outermost index, col4 becomes inner index. Values of col3, col4 become the index values.</pre>
-------------------------------	--

* “reset_index” does the opposite of “set_index”, the hierarchical index are moved into columns.

‡ By default, 'col3' and 'col4' will be removed from the DF, though you can leave them by option : “drop = False”.

MISSING DATA

Python	NaN - np.nan (not a number)
Pandas *	NaN or python built-in None mean missing/NA values

* Use `pd.isnull()`, `pd.notnull()` or `series1/df1.isnull()` to detect missing data.

FILTERING OUT MISSING DATA

`dropna()` returns with ONLY non-null data, source data NOT modified.

<pre>df1.dropna() # drop any row containing missing value</pre>
<pre>df1.dropna(axis = 1) # drop any column containing missing values</pre>

<pre>df1.dropna(how = 'all') # drop row that are all missing</pre>
<pre>df1.dropna(thresh = 3) # drop any row containing < 3 number of observations</pre>

FILLING IN MISSING DATA

<pre>df2 = df1.fillna(0) # fill all missing data with 0 df1.fillna(inplace = True) # modify in-place Use a different fill value for each column : df1.fillna({'coll' : 0, 'col2' : -1})</pre>
Only forward fill the 2 missing values in front : <pre>df1.fillna(method = 'ffill', limit = 2)</pre>
i.e. for column1, if row 3-6 are missing. so 3 and 4 get filled with the value from 2, NOT 5 and 6.

ESSENTIAL FUNCTIONALITY

INDEXING (SLICING/SUBSETTING) †

- † Same as 'NdArray': In INDEXING: 'view' of the source array is returned.
- † Endpoint is inclusive in pandas slicing with labels: `series1['a':'c']` where Python slicing is NOT. Note that pandas non-label (i.e. integer) slicing is still non-inclusive.

Index by Column(s)	<code>df1['col1']</code> <code>df1[['col1', 'col3']]</code>
Index by Row(s)	<code>df1.ix['row1']</code> <code>df1.ix[['row1', 'row3']]</code>
Index by Both Column(s) and Row(s)	<code>df1.ix[['row2', 'row1'], 'col3']</code>
Boolean Indexing	<code>df1[[True, False]]</code> <code>df1[df1['col2'] > 6]</code> * # returns df that has col2 value > 6

- Note that `df1['col2'] > 6` returns a boolean Series, with each True/False value determine whether the respective row in the result.
- Avoid integer indexing since it might introduce subtle bugs (e.g. `series1[-1]`).
- If have to use position-based indexing, use `"iget_value()"` from Series and `"irow/icol()"` from DF instead of integer indexing.

DROPPING ROWS/COLUMNS

Drop operation returns a new object (i.e. DF) :

Remove Row(s) (axis = 0 is default)	<code>df1.drop('row1')</code> <code>df1.drop(['row1', 'row3'])</code>
Remove Column(s)	<code>df1.drop('col2', axis = 1)</code>

REINDEXING

Create a new object with rearranging data conformed to a new index, introducing missing values if any index values were not already present.

Change df1 Date Index Values to the New Index Values	<code>date_index = pd.date_range('01/23/2010', periods = 10, freq = 'D')</code>
(ReIndex default is row index)	<code>df1.reindex(date_index)</code>
Replace Missing Values (NaN) with 0	<code>df1.reindex(date_index, fill_value = 0)</code>
ReIndex Columns	<code>df1.reindex(columns = ['a', 'b'])</code>
ReIndex Both Rows and Columns	<code>df1.reindex(index = [...], columns = [...])</code>
Succinct ReIndex	<code>df1.ix[[...], [...]]</code>

ARITHMETIC AND DATA ALIGNMENT

- `df1 + df2` : For indices that don't overlap, internal data alignment introduces NaN.

1, Instead of NaN, replace with 0 for the indice that is not found in th df: <code>df1.add(df2, fill_value = 0)</code>
2, Useful Operations : <code>df1 - df1.ix[0]</code> # subtract every row in df1 by first row

SORTING AND RANKING

Sort Index/Column †

- `sort_index()` returns a new, sorted object. Default is "ascending = True".
- Row index are sorted by default, "axis = 1" is used for sorting column.

- † Sorting Index/Column means sort the row/column labels, not sorting the data.

Sort Data

Missing values (np.nan) are sorted to the end of the Series by default

Series Sorting	<code>sortedS1 = series1.order()</code> <code>series1.sort()</code> # In-place sort
DF Sorting	<code>df1.sort_index(by = ['col2', 'col1'])</code> # sort by col2 first then col1

Ranking

Break rank ties by assigning each tie-group the mean rank. (e.g. 3, 3 are tie as the 5th place; thus, the result is 5.5 for each)

Output Rank of Each Element (Rank start from 1)	<code>series1.rank()</code> <code>df1.rank(axis = 1)</code> # rank each row's value
--	---

FUNCTION APPLICATIONS

NumPy works fine with pandas objects : `np.abs(df1)`

Applying a Function to Each Column or Row (Default is to apply to each column : axis = 0)	<code>f = lambda x: x.max() - x.min()</code> # return a scalar value <code>def f(x): return Series([x.max(), x.min()])</code> # return multiple values <code>df1.apply(f)</code>
Applying a Function Element-Wise	<code>f = lambda x: '%.2f' % x</code> <code>df1.applymap(f)</code> # format each entry to 2-decimals

UNIQUE, COUNTS

- It's **NOT** mandatory for index labels to be unique although many functions require it. Check via : `series1/df1.index.is_unique`
- `pd.value_counts()` returns value frequency.

DATA AGGREGATION AND GROUP OPERATIONS

Categorizing a data set and applying a function to each group, whether an aggregation or transformation.

- Note Aggregation of "Time Series" data - please see Time Series section. Special use case of "groupby" is used - called "resampling".

GROUPBY (SPLIT-APPLY-COMBINE)

- Similar to SQL groupby

Compute Group Mean	<code>df1.groupby('col2').mean()</code>
GroupBy More Than One Key	<code>df1.groupby([df1['col2'], df1['col3']]).mean()</code> # result in hierarchical index consisting of unique pairs of keys
"GroupBy" Object : (ONLY computed intermediate data about the group key - df1['col2'])	<code>grouped = df1['col1'].groupby(df1['col2'])</code> <code>grouped.mean()</code> # gets the mean of each group formed by 'col2' # select 'col1' for aggregation :
Indexing "GroupBy" Object	<code>df1.groupby('col2')['col1']</code> or <code>df1['col1'].groupby(df1['col2'])</code>

- Note Any missing values in the group are excluded from the result.

1. Iterating over GroupBy object

"GroupBy" object supports iteration : generating a sequence of 2-tuples containing the group name along with the chunk of data.

<code>for name, groupdata in df1.groupby('col2'):</code> # name is single value, groupdata is filtered DF contains data only match that single value. <code>for (k1, k2), groupdata in df1.groupby(['col2', 'col3']):</code> # If groupby multiple keys : first element in the tuple is a tuple of key values.

Convert Groups to Dict	<code>dict(list(df1.groupby('col2')))</code> # col2 unique values will be keys of dict
Group Columns by "dtype"	<code>grouped = df1.groupby([df1.dtypes, axis = 1])</code> <code>dict(list(grouped))</code> # separates data into different types

2. Grouping with functions

Any function passed as a group key will be called once per (default is row index) value, with the return values being used as the group names. (This assumes row index are named)

<code>df1.groupby(len).sum()</code> # returns a DF with row index that are length of the names. Thus, names of same length will sum their values. Column names retain.

DATA AGGREGATION

Data aggregation means any data transformation that produces **scalar** values from arrays, such as "mean", "max", etc.

Use Self-Defined Function	<code>def func1(array): ...</code> <code>grouped.agg(func1)</code>
Get DF with Column Names as Function Names	<code>grouped.agg([mean, std])</code>
Get DF with Self-Defined Column Names	<code>grouped.agg([('col1', mean), ('col2', std)])</code>
Use Different Function Depending on the Column	<code>grouped.agg({'col1': [min, max], 'col3': sum})</code>

GROUP-WISE OPERATIONS AND TRANSFORMATIONS

Agg() is a special case of data transformation, aka reduce a **one-dimensional array to scalar**.

Transform() is a specialized data transformation :

- It applies a function to each group, if it produces a scalar value, the value will be placed in **every row** of the group. Thus, if DF has 10 rows, after "transform()", there will be still 10 rows, each one with the scalar value from its respective group's value from the function.

- The passed function must either produce a scalar value or a transformed array of same size.

General purpose transformation : apply()

<code>df1.groupby('col2').apply(your_func1)</code> # your func ONLY need to return a pandas object or a scalar. # Example 1 : Yearly Correlations with SPX # "close_price" is DF with stocks and SPX closed price columns and dates index <code>returns = close_price.pct_change().dropna()</code> <code>by_year = returns.groupby(lambda x : x.year)</code> <code>spx_corr = lambda x : x.corrwith(x['SPX'])</code> <code>by_year.apply(spx_corr)</code> # Example 2 : Exploratory Regression <code>import statsmodels.api as sm</code> <code>def regress(data, y, x):</code> <code>Y = data[y]; X = data[x]</code> <code>X['intercept'] = 1</code> <code>result = sm.OLS(Y, X).fit()</code> <code>return result.params</code> <code>by_year.apply(regress, 'AAPL', ['SPX'])</code>

Created by Arianne Colton and Sean Chen

www.datasciencefree.com

Based on content from "Python for Data Analysis" by Wes McKinney

Updated: August 22, 2016

DATA WRANGLING : MERGE, RESHAPE, CLEAN, TRANSFORM

COMBINING AND MERGING DATA

1. **pd.merge()** aka database “join” : connects rows in DF based on one or more keys.

- Merge via Column (Common)

```
df3 = pd.merge(df1, df2, on = 'col2') *  
# INNER join is default Or use option : how = 'outer/  
left/right'  
# the indexes of df1 and df2 are discarded in df3
```

Use ALL overlapping column names as the keys to merge. Good practice is to specify the keys :
on = ['col2', 'col3'].
* If different key name in df1 and df2, use option :
left on='lkey', right on='rkey'

- Merge via Row (Uncommon)

```
df3 = pd.merge(df1, df2, left_index =  
True, right_index = True)  
# Use indexes as merge key : aka rows with same index  
value are joined together.
```

2. **pd.concat()** : glues or stacks objects along an axis (default is along “rows : axis = 0”).

```
df3 = pd.concat([df1, df2], ignore_index  
= True) # ignore_index = True : Discard indexes in df3  
# If df1 has 2 rows, df2 has 3 rows, then df3 has 5 rows
```

3. **combine_first()** : combine data with overlap, patching missing value.

```
df3 = df1.combine_first(df2)  
# df1 and df2 indexes overlap in full or part. If a row NOT  
exist in df1 but in df2, it will be in df3. If row1 of df1 and  
row3 of df2 have the same index value, but row1's col3  
value is NA, df3 get this row with the col3 data from df2
```

RESHAPING AND PIVOTING

1. Reshaping with Hierarchical Indexing

```
series1 = df1.stack()  
# Rotates (innermost level *) columns to rows as innermost  
index level, resulted in Series with hierarchical index.  
df1 = series1.unstack()  
# Rotates (innermost level *) rows to columns as innermost  
column level.
```

Default is to stack/unstack innermost level. If
* want a different level, i.e. stack(level =
0) - the outermost level.

Note : Unstacking might introduce missing data if
not all of the values in the level aren't found in each
of the subgroups. Stacking filters out missing data
by default, i.e. data.unstack().stack()

2. Pivoting

- Common format of storing multiple “time series” in
databases and CSV is :

Long/Stacked Format :	"date, stock_name, price"
-----------------------	---------------------------

- However, a DF with these 3 columns data like above
will be difficult to work with. Thus, “wide” format
is preferred : 'date' as row index, 'stock_name' as
columns, 'price' as DF data values.

```
pivotedDf2 = df1.pivot('date', 'stock_  
name', 'price')  
# Example pivotedDf2 :  
#                AAPL  IBM   JD  
# 2003-06-01  120.2  100.1  20.8
```

COMMON OPERATIONS

1. Removing Duplicate Rows

```
series1 = df1.duplicated() # Boolean series1  
indicating whether each row is a duplicate or not.  
df2 = df1.drop_duplicates() # Duplicates has  
been dropped in df2.
```

2. Add New Column Based On Value of Column(s)

```
df1['newCol'] = df1['col2'].map(dict1)  
# Maps col2 value as dict1's key, gets dict1's value  
df1['newCol'] = df1['col2'].map(func1)  
# Apply a function to each col2 value
```

3. Replacing Values

```
# Replace is NOT In-Place  
df2 = df1.replace(np.nan, 100)  
# Replace Multiple Values At Once  
df2 = df1.replace([-1, np.nan], 100)  
df2 = df1.replace([-1, np.nan], [1, 2])  
# Argument Can Be A Dict As Well  
df2 = df1.replace({'-1': 1, np.nan : 2})
```

4. Renaming Axis Indexes

Convert Index to Upper Case	df1.index = df1.index. map(str.upper)
Rename 'row1' to 'newRow1'	df2 = df1.rename(index = { 'row1' : 'newRow1' }, columns = str.upper) # Optionally inplace = True

5. Discretization and Binning

- Continuous data is often discretized into “bins” for
analysis.

```
# Divide Data Into 2 Bins of Number (18 - 26], (26 - 35]  
# ']' means inclusive, '[' is NOT inclusive  
bins = [18, 26, 35]  
cat = pd.cut(array1, bins, labels=[])  
# cat is "Categorical" object.  
pd.value_counts(cat)  
cat = pd.cut(array1, numofBins) # Compute  
equal-length bins based on min and max values in array1  
cat = pd.qcut(array1, numofBins) # Bins the  
data based on sample quantiles - roughly equal-size bins
```

6. Detecting and Filtering Outliers

- any() test along an axis if any element is “True”.
Default is test along column (axis = 0).

```
df1[(np.abs(df1) > 3).any(axis = 1)]  
# Select all rows having a value > 3 or < -3.  
# Another useful function : np.sign() returns 1 or -1.
```

7. Permutation and Random Sampling

```
randomOrder = np.random.permutation(df1.  
shape[0])  
df2 = df1.take(randomOrder)
```

8. Computing Indicator/Dummy Variables

- If a column in DF has “K” distinct values, derive a
“indicator” DF containing K columns of 0s and 1s.
1 means exist, 0 means NOT exist.

```
dummyDf = pd.get_dummies(df1['col2'],  
prefix = 'col-') # Add prefix to the K column names
```

GETTING DATA

TEXT FORMAT (CSV)

```
df1 = pd.read_csv(file/URL/file-like-object,  
sep = ',', header = None)
```

Type-Inference : do NOT have to specify which columns are
numeric, integer, boolean or string.

In Pandas, missing data in the source data is usually empty
string, NA, -1, #N/A or NULL. You can specify missing values
via option i.e. : na_values = ['NULL'].

Default delimiter is comma.

Default is first row is the column header.

```
df1 = pd.read_csv(.., names = [..])
```

Explicitly specify column header, also imply first row is data
df1 = pd.read_csv(.., names = [..,
'date'], index_col = 'date')

Want 'date' column to be row index of the returned DF

```
df1.to_csv(filepath/sys.stdout, sep = ',', )
```

Missing values appear as empty strings in the output. Thus,
It is better to add option i.e. : na_rep = 'NULL'

Default is row and column labels are written. Disabled by
options : index = False, header = False

JSON (JAVASCRIPT OBJECT NOTATION) DATA

One of the standard formats for sending data by HTTP
request between web browsers and other applications.
It is much more flexible data format than tabular text from
like CSV.

Convert JSON string to Python form	data = json.load(jsonObj)
Convert Python object to JSON	asJson = json.dumps(data)
Create DF from JSON	df1 = pd.DataFrame(data['name'], columns = ['field1'])

XML AND HTML DATA

HTML :

```
doc = lxml.html.  
parse(urlopen('http://..')).getroot()  
tables = doc.findall('..//table')  
rows = tables[1].findall('..//tr')
```

XML :
lxml.objectify.parse(open(filepath)).
getroot()

DESCRIPTIVE STATISTICS METHODS †

† Compared with equivalent methods of ndarray,
descriptive statistics methods in Pandas are built
from the ground up to exclude missing data.

† NA (i.e. NaN) values are excluded. This can be
disabled using the “skipna = False” option.

Column Sums (Use axis = 1 to sum over rows)
series1 = df1.sum()
Returns Index Labels Where Min/Max Values are Attained
df1.idxmin() or df1.idxmax()
Multiple Summary Statistics (i.e. count, mean, std)
On Non-Numeric Data, Alternate Statistics (i.e. count, unique)
df1.describe()

CORRELATION AND COVARIANCE

- cov(), corr()
- corrwith() - pairwise correlations : aka compute
a DF with a Series. If input is not Series, but another
DF, it will compute the correlations of matching column
names. i.e. returns.corrwith(volumes)

```
# Example : Correlation  
import pandas_datareader.data as web  
data = {}  
for ticker in ['AAPL', 'JD'] :  
    data[ticker] = web.get_data_  
yahoo(ticker, '1/1/2000', '1/1/2010')  
prices = pd.DataFrame({ticker : d['Adj  
Close'] for ticker, d in data.iteritems()})  
volumes = ..  
returns = prices.pct_change()  
returns.AAPL.corr(returns.JD)  
# Series corr() computes correlation of overlapping, non-NA,  
aligned-by-index values in two Series.
```

TIME SERIES

- **Python** standard library data types for date and time : "datetime", "time", "calendar". †
- **Pandas** data type for date and time : "Timestamp". *

Convert String to Date/Time	from datetime import datetime datetime.strptime('8/8/2008', '%m/%d/%Y')
Get Time Now	<code>now = datetime.now()</code>
Date/Time Arithmetic	from datetime import timedelta datetime(2011, 1, 8) + timedelta(12) => 2011-01-20 # Timedelta represents temporal difference between two datetime objects.
Convert String to Pandas Timestamp Type	<code>timestamps = pd.to_datetime(['8/8/2008', ..])</code> # NaT (Not a Time) is Pandas NA Value for Timestamp Data <code>pd.to_datetime('') => NaT</code> <code>pd.isnull(NaT) => True</code> # Missing value (i.e. empty string)

- † "datetime" is widely used, it stores both the date and time down to microsecond.
- * "Timestamp" object can be substituted anywhere you would use "datetime" object.

PANDA TIME SERIES

Create Time Series
<code>ts1 = pd.Series(np.random.randn(8), index = [datetime(2011, 1, 2), ..])</code>
<code>ts1 = pd.Series(..., index = pd.date_range('1/1/2000', periods = 1000))</code>
<code>ts1.index</code> is "DatetimeIndex" Panda class

- Index value `ts1.index[0]` is Panda "Timestamp" object which stores timestamp using NumPy's "datetime64" type at the nanosecond resolution. Further, Timestamp class stores the frequency information as well as timezone.
- † `ts1.index.dtype => datetime64[ns]`

Indexing (Slicing/Subsetting)

Argument can be a string date, datetime or Timestamp.

Select Year of 2001	<code>ts1['2001']</code> <code>dfl.ix['2001']</code>
Select June 2001	<code>ts1['2001-06']</code>
Select From 2001-01-01 to 2001-08-01	<code>ts1['1/1/2001':'8/1/2001']</code>
Select From 2001-01-08 On	<code>ts1[datetime(2001, 1, 8):]</code>

Common Operations

Get Time Series Data Before 2011-01-09	<code>ts1.truncate(after = '1/8/2011')</code>
--	---

DATE RANGES, FREQUENCIES AND SHIFTING

Generic time series in Pandas are assumed to be irregular, aka have no fixed frequency. However, we prefer to work with fixed frequency, i.e. daily, monthly, etc.

Take a Look at "Resampling" Section	# Convert to Fixed Daily Frequency. # Introduce Missing Value (NaN) If Needed <code>ts1.resample('D', how = ..)</code>
-------------------------------------	--

1. Frequencies and Date Offsets

- Frequencies in Pandas are composed of a base frequency and a multiplier. Base frequencies are typically referred to by a string alias, like 'M' for monthly or 'H' for hourly.

```
freq = '4H'
freq = '1h30min'
# Standard US equity option monthly expiration, every third Friday of the month : freq = 'WOM-3FRI'
```

2. Generating Date Ranges

Default Frequency is Daily	<code>pd.date_range(begin, end)</code> Or <code>pd.date_range(begin or end, periods = n)</code> # Option <code>freq = 'BM'</code> means last business day at end of the month
----------------------------	--

3. Shifting (Leading and Lagging) Data

- Shifting refers to moving data backward and forward through time.
- Series and DF "shift()" does naive shift, aka index does not shift, only value. *

```
# ts1 is Daily Data
ts1.shift(1) # move yesterday's value to today, today value to tomorrow, etc.

# ts1 is Any Time Series Data. Shift Data By 3 Days
ts1.shift(3, freq = 'D') Or
ts1.shift(1, freq = '3D')

# Common Use of Shift : To Computer % Change
ts1 / ts.shift(1) - 1
```

- * In the return result from shift(), some data value might be NaN.

- Other ways to shift data :

```
from pandas.tseries.offsets import Day, MonthEnd
datetime(2008, 8, 8) + 3*Day() => 2008-08-11
datetime(2008, 8, 8) + MonthEnd(2) => 2008-09-30
MonthEnd().rollforward(datetime(2008, 8, 8)) => 2008-08-31
```

TIME ZONE HANDLING

- **Daylight saving time (DST)** transitions are a common source of complication.
- **UTC** is the current international standard. Time zones are expressed as offsets from UTC. *

- * NY is 4 hours behind UTC during daylight saving time and 5 hours the rest of the year.

1. Python Time Zone (From 3rd-party pytz library)

Get List of Timezone Names	<code>pytz.common_timezones</code>
Get a Timezone Object	<code>pytz.timezone('US/Eastern')</code>

2. Localization and Conversion

Time Series By Default is Time Zone Naive	<code>ts1.index.tz => None</code>
Specify Time Zone When Create Time Series	Use option : <code>tz = 'UTC'</code> in <code>pd.date_range()</code>
Localization From Naive	<code>ts1_utc = ts1.tz_localize('UTC')</code>
Convert to Another Time Zone Once Time Series Been Localized	<code>ts1_eastern = ts1_utc.tz_convert('US/Eastern')</code>

3. ** Time Zone-aware Timestamp Objects

```
stamp_utc = pd.Timestamp('2008-08-08 03:00', tz = 'UTC')
stamp_eastern = stamp_utc.tz_convert(...)
```

Panda's Time Arithmetic - Daylight Savings Time Transitions Are Respected :

```
stamp = pd.Timestamp('2012-11-04 00:30', tz = 'US/Eastern') => 2012-11-04-00:30:00 -400 EDT
stamp + 2 * Hour() => 2012-11-04-01:30:00 -500 EST
```

- ** If two time series with different time zones are combined, i.e. `ts1 + ts2`, the timestamps will auto-align with respect to time zone. The result will be in UTC.

RESAMPLING

Process of converting a time series from one frequency to another frequency :

- 1. **Downsampling** - Aggregating higher frequency data to lower frequency.

```
* ts1.resample('M', how = 'mean')
=> Index: 2000-01-31, 2000-02-29, ...

ts1.resample('M', ..., kind = 'period')
# 'period' - Use time-span representation
=> Index: 2000-01, 2000-02, ...

# ts1 is one minute data of value 1 to 100 of time :
00:00:00, 00:01:00, ...
ts1.resample('5min', how = 'sum') =>
00:00:00    15 (aka : 1 + 2 + 3 + 4 + 5)
00:05:00    40

# Default is left bin edge is inclusive, thus 00:00:00 value is included in the 00:00:00 to 00:05:00 interval.

# Option : closed = 'right' change interval to right inclusive. Also include option label = 'right' as well :
00:00:00    1
00:05:00    20 (aka : 2 + 3 + 4 + 5 + 6)
```

```
ts1.resample('5min', how = 'ohlc')
# returns a DF with 4 columns - open, high, low, close
```

- * Alternate way to downsample : `ts1.groupby(lambda x : x.month).mean()`

- 2. **Upsampling and Interpolation** * - Interpolate low frequency to higher frequency. By default missing values (NaN) are introduced.

```
dfl.resample('D', fill_method = 'ffill')
# Forward fills values : i.e. missing value index such as index 3 will copy value from index 2.
```

- * Interpolation will ONLY apply row-wise.

TIME SERIES PLOTTING

Example : Source Data Format - First Column is Date.

Use first column as the Index, then parse the index values as Date.

```
prices = pd.read_csv(..., parse_date = True, index_col = 0)
px = prices[['AAPL', 'IBM']]
px = px.resample('B', fill_method = 'ffill')
px['AAPL'].plot()
px['AAPL'].ix['01-2008':'03-2012'].plot()
px.ix['2008'].plot()
```

MOVING WINDOW FUNCTIONS

Like other statistical functions, these functions also automatically exclude missing data.

```
pd.rolling_mean(px.AAPL, 200).plot()
pd.rolling_std(px.AAPL.pct_change(), 22, min_periods = 20).plot()
pd.rolling_corr(px.AAPL.pct_change(), px.IBM.pct_change(), 22).plot()
```

PERFORMANCE

- Since "Timestamps" is represented as 64-bit integers using NumPy's datetime64 type, it means for each data point, there is an associated 8 bytes of memory per timestamp.
- **"Creating views"** on existing time series or DF do not cause any more memory to be used.
- Indexes for lower frequencies (daily and up) are stored in a **central cache**, so any fixed-frequency index is a **view** on the date cache. Thus, low-frequency indexes memory footprint is not significant.
- Performance-wise, Pandas has been highly optimized for data alignment operations (i.e. `ts1 + ts2`) and resampling.

Created by Arianne Colton and Sean Chen

www.datasciencefree.com

Based on content from "Python for Data Analysis" by Wes McKinney

Updated: August 22, 2016